

NutriGuide — Full Project Summary

What is NutriGuide?

NutriGuide is a **full-stack personalized diet & nutrition tracking web application** for Indian users. It lets users register, set their health goals, get personalized Indian meal/recipe recommendations, log daily meals, and track nutrition analytics — all with role-based access (Admin vs. User).

Tech Stack

Layer	Technology
Frontend	React 19, Vite, React Router DOM v7, Recharts, Axios, Lucide React
Backend	Node.js, Express.js
Database	MySQL (via mysql2 pool)
Auth	JWT (JSON Web Tokens) + bcrypt password hashing
Styling	Vanilla CSS

Project Architecture

```
NutriGuide/  
  frontend/  
    src/  
      main.jsx          ← App entry point  
      App.jsx           ← Router + Route Guards  
      index.css         ← Global styles  
      context/  
        AuthContext.jsx ← Global auth state  
      services/  
        api.js          ← Axios instance (auto-attaches JWT)  
      components/  
        Navbar.jsx      ← Navigation bar  
      pages/  
        LandingPage.jsx  
        Login.jsx  
        Register.jsx  
        Dashboard.jsx  
        Meals.jsx  
        Progress.jsx  
        Analytics.jsx
```

```

    Profile.jsx
    Help.jsx
    AdminPanel.jsx

backend/
  server.js           ← Express app + route mounting
  config/db.js        ← MySQL pool connection
  middlewares/auth.js ← verifyToken + isAdmin guards
  routes/
    authRoutes.js
    recipeRoutes.js
    mealRoutes.js
    helpRoutes.js
  controllers/
    authController.js
    recipeController.js
    mealController.js
    helpController.js

```

Authentication — How It Works

Authentication uses the **JWT (JSON Web Token)** pattern — stateless and secure.

Flow Diagram

```

User fills Login form
↓
POST /api/auth/login
↓
Backend: Find user by email in DB
↓
bcrypt.compare(password, hashedPassword)
↓
If match → jwt.sign({ id, role }, SECRET, { expiresIn: '24h' })
↓
Token sent back to Frontend
↓
Frontend: localStorage.setItem('token', token)
↓
AuthContext.login() sets user state globally
↓
App.jsx ProtectedRoute checks user → grants access

```

Key Files & Their Roles

backend/middlewares/auth.js This is the **security gatekeeper** for all protected API routes.

verifyToken middleware:

1. Reads Authorization header → "Bearer <token>"
2. Calls `jwt.verify(token, JWT_SECRET)`
3. If valid → sets `req.userId` and `req.userRole` → calls `next()`
4. If invalid → returns 401 Unauthorized

isAdmin middleware:

1. Checks `req.userRole === 'admin'`
2. If not admin → returns 403 Forbidden
3. If admin → calls `next()`

backend/controllers/authController.js

Function	HTTP Method	Route	Description
register	POST	/api/auth/register	Creates new user. Hashes password with bcrypt (salt=10). Checks for duplicate email.
login	POST	/api/auth/login	Verifies credentials, returns JWT token + user data
getProfile	GET	/api/auth/profile	Returns logged-in user's full profile (requires token)
updateProfile	PUT	/api/auth/profile	Updates user details like weight, diet preference, goals (requires token)

frontend/src/context/AuthContext.jsx The **global auth state manager** on the frontend: - On app load → checks `localStorage` for token → auto-fetches profile from backend → sets user state - `login(userData, token)` → saves token + sets user in state - `logout()` → clears token + nullifies user state - `updateUser(data)` → merges new data into existing user state (used after profile update)

frontend/src/services/api.js A centralized **Axios instance** with an **interceptor**: - Base URL: `http://localhost:5000/api` - **Interceptor**: Before every request, automatically reads the JWT from `localStorage` and attaches it as **Authorization: Bearer <token>** header → so you never need to manually add the header in any page

CRUD Operations — Explained by Resource

NutriGuide has CRUD operations across 3 main resources:

1. User / Profile (Auth Module)

Operation	CRUD	HTTP	Endpoint	Who Can Access
Register new user	Create	POST	<code>/api/auth/register</code>	Public
Login	Read (auth)	POST	<code>/api/auth/login</code>	Public
Get my profile	Read	GET	<code>/api/auth/profile</code>	logged-in user
Update my profile	Update	PUT	<code>/api/auth/profile</code>	logged-in user
Delete user	—	—	—	Not implemented

What you can update in profile: name, age, weight, height, diet_preference, health_goal, allergies, target_weight

2. Recipes (Recipe Module)

Operation	CRUD	HTTP	Endpoint	Who Can Access
Get all recipes	Read	GET	<code>/api/recipes/</code>	Public
Get personalized recommendations	Read	GET	<code>/api/recipes/recommendations</code>	logged-in user
Create a new recipe	Create	POST	<code>/api/recipes/</code>	Admin only

Operation	CRUD	HTTP	Endpoint	Who Can Access
Delete a recipe	Delete	DELETE	/api/recipes/ id	Admin only
Update recipe	—	—	—	Not implemented

How Recommendations Work (Smart Read):

1. Fetch user's diet_preference, allergies, health_goal, weight, target_weight
2. Filter recipes by diet_preference (vegan / vegetarian / non-veg)
3. Remove recipes containing allergenic ingredients
4. Sort by calories based on goal:
 - lose_weight → low calorie first (500 cal)
 - gain_muscle → high calorie first (600 cal)
 - maintain → random shuffle
5. Return top 10 results

3. Meal Logs (Meal Tracking Module)

Operation	CRUD	HTTP	Endpoint	Who Can Access
Log a meal	Create	POST	/api/meals/	Logged-in user
Get today's meals	Read	GET	/api/meals/daily?date=YYYY-MM-DD	Logged-in user
Get nutrition analytics	Read	GET	/api/meals/analysis?days=7	Logged-in user
Reset/delete today's logs	Delete	DELETE	/api/meals/daily?date=YYYY-MM-DD	Logged-in user
Update a meal log	—	—	—	Not implemented

Page-by-Page Map

Page	Route	Purpose
LandingPage	/	Marketing landing page for guests
Login	/login	User login form

Page	Route	Purpose
Register	<code>/register</code>	User signup form with health details
Dashboard	<code>/dashboard</code>	BMI, calorie summary, daily overview
Meals	<code>/meals</code>	Browse + log recommended Indian recipes
Progress	<code>/progress</code>	Weight tracking over time
Analytics	<code>/analytics</code>	Charts for calories, protein, carbs, fat (Recharts)
Profile	<code>/profile</code>	View & edit personal health details
Help	<code>/help</code>	FAQ / support section
AdminPanel	<code>/admin</code>	Admin-only: manage recipes

Route Guard Summary (Frontend)

`ProtectedRoute` → Must be logged in → Otherwise → redirect to /
`AuthRedirect` → If already logged in → redirect to `/dashboard` (prevents accessing `/login`)
`HomeRedirect` → Logged-in user → `/dashboard` | Guest → `LandingPage`
`ProtectedRoute requireAdmin` → Must be logged in AND `role='admin'` → otherwise `/dashboard`

Database Tables

Table	Purpose
<code>users</code>	Stores user accounts, health data, preferences
<code>recipes</code>	Stores all Indian meal recipes with nutrition info
<code>meal_logs</code>	Links users to recipes they ate on specific dates

Missing CRUD Operations (Gaps)

[!NOTE] These CRUD operations are **not yet implemented** and could be added:

Resource	Missing
Users	Delete account
Recipes	Update/Edit recipe (Admin)
Meal Logs	Update a specific log entry
Progress	Dedicated weight-log CRUD